Predicting Consecutive-Record Travel Intervals of Delhi City Buses Using Node-Level Timestamp Data and Machine Learning

Muskan Sharma

Department of Computer Science, Ramanujan College, University of Delhi, New Delhi, India | Exam Roll No.: 24020570020 | College Roll No.: 24570040

Course: Machine Learning (B.Sc. (Hons.) Computer Science) | Supervisor: Mr. Nikhil | Problem No. 49 (Transport) | Article Type: Original Research Article

Abstract. This paper examines how much can be learned from the bus-record files supplied for Problem 49. The aim is to estimate the time between two consecutive observations of the same vehicle at adjacent node codes. We analyse 924,375 timestamped node-passing records of 202 buses supplied as 184 Excel workbooks covering three months and two route-group folders. The files contain a node code, vehicle identifier, direction and time of day, but no verified stop map, GPS trace or calendar date. The target is therefore a consecutive-record interval, not a validated stop-to-stop ETA. Instead we predict the time between two consecutive records of the same bus at adjacent node codes (867,574 samples, median 1.33 min). Five regressors (linear, polynomial, k-NN, decision tree, MLP) were compared with three baselines using a month-folder-based holdout split (train: Months 1–2, test: Month 3). The MLP gave the lowest test error (MAE 0.596 min, RMSE 1.426 min, R^2 0.474), but the decision tree was within 0.002 min RMSE and the linear model reached R^2 0.422. Compared with a segment-wise historical mean, the best model lowered RMSE by 7.0% and MAE by 13.1%. Predicting the previous interval was worse than predicting the training mean. The results suggest that these fields contain a useful but limited signal. A passenger-facing ETA system would still require verified stop locations, GPS observations and timetable information.

Keywords: bus travel time; Delhi; regression; k-NN; decision tree; MLP; month-folder holdout validation; SDG 9; SDG 11

1 Introduction

When buses arrive unpredictably, commuters lose time at stops and many switch to private vehicles, which increases congestion and emissions. Accurate travel-time estimates are therefore linked to SDG 11 (Sustainable Cities and Communities) and SDG 9 (Industry, Innovation and Infrastructure). Our allocated problem (No. 49) asks for bus travel-time prediction for Delhi from time of day, route, stop sequence and position information.

The supplied files are much simpler than a complete public-transport feed. Each row records a vehicle identifier, a numbered node, a direction and a time of day. Coordinates, verified stop names, schedules and calendar dates are not included. We therefore define a narrower and honest target, the *consecutive-record interval*, and keep it separate from true stop-to-stop ETA. The work has four practical parts: (i) a repeatable process for converting 184 workbooks into interval samples; (ii) a month-folder holdout comparison of five syllabus-level regressors and three baselines; (iii) an error analysis by hour of day; and (iv) a clear statement of what the data cannot support. Sections 2–3 cover related work and data, Sections 4–5 the method and set-up, Section 6 the results, and Section 7 the discussion.

2 Related Work

Bus arrival time (BAT) prediction has a large literature. Singh and Kumar [1] review artificial-intelligence methods applied to BAT, and Abdi and Amrit [2] give a taxonomy of travel- and arrival-time prediction methods on road networks, discussing the factors that affect accuracy. Agafonov and Yumaganov [3] compare machine-learning methods, including neural networks and support-vector regression, for public-transport arrival time and note that the best choice depends on the input data used. Most such studies use GPS positions, schedules or traffic information, which are not available in our files. Because our observations are ordered in time, we avoid random shuffling and evaluate on a later period, in line with the cautions about cross-validation for time-ordered data discussed by Bergmeir and Benítez [4]. All models were implemented with scikit-learn [5]; the algorithms follow standard texts [6]. The k-NN rule goes back to Cover and Hart [7], regression trees to Breiman et al. [8], and the MLP is trained with the Adam optimiser [9].

3 Dataset and Preprocessing

3.1 Source and description

The data were supplied as three archives (Month_1, Month_2, Month_3) containing 184 Excel workbooks, one per day, inside two sub-folders named 425 and 534. The original provider could not be independently confirmed. The allocation sheet points to the Delhi Open Transit Data portal, but the files we received are not GTFS tables and we did not verify their provenance, so we do not attribute them to any agency. We also do not know whether the folder names are bus route numbers; we call them *route-group folders*. All workbooks were read programmatically (no unreadable files). After removing the index column, every workbook has four fields (Table 1). In total there are 924,375 records, 202 distinct bus identifiers, 56 node codes and two directions (up, down). Records fall between 04:48 and 22:00. Folder 425 has 41 node codes and 46 buses; folder 534 has 56 node codes and 156 buses.

Table 1. Fields in the supplied workbooks and their use in this study.

Field	Type	Distinct	Missing	Use
Node	integer	56	0	segment position (from-node)
Number	text	202	0	groups records by bus
Route	up / down	2	0	direction feature
Time	time string	–	0	hour features; interval target
Folder path	folder names	3 / 2 / 1–31	–	split; folder feature

Table 2. Summary of the two route-group folders (all three months).

Folder	Records	Buses	Nodes	Samples	Mean (min)	Median (min)
425	185,788	46	41	171,177	2.16	1.50
534	738,587	156	56	696,397	1.78	1.33

3.2 Missing values, duplicates and interval construction

There are no missing cells and no duplicate records (Fig. 2), so the main quality problems lie in the structure of the sequences. Within one bus, direction and day, the node code usually advances by one; 6.1% of consecutive pairs are not adjacent (a skipped node, a new trip or a direction change) and were excluded. Of the adjacent pairs, 43 had a non-positive interval (time wrap-around between trips) and 800 exceeded 30 minutes, which we treat as layovers or logging gaps rather than travel. This leaves **867,574** samples (294,575 / 291,103 / 281,896 in Months 1/2/3). The 30-minute cut-off is a judgement call; intervals are short (Fig. 5), so it removes very little.

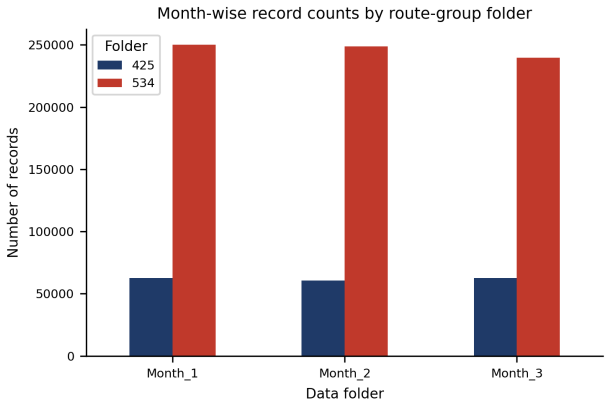


Fig. 1. Number of records per month folder and route-group folder. Folder 534 contributes about four times as many records as folder 425 in every month.

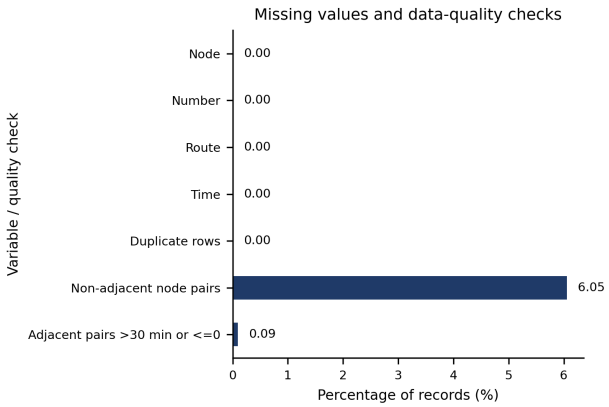


Fig. 2. Missing-value and data-quality checks. The raw fields are complete; the only notable issue is non-adjacent node pairs.

3.3 Exploratory analysis

Time of day (Fig. 3): records rise sharply after 05:00, peak in the 08:00 hour and decline through the evening. Node frequency (Fig. 4) is fairly even up to the high 30s and lower for higher codes, and codes above 41 occur only in folder 534; a few nodes (around code 22 and codes 27–29) have clearly fewer records, which may mean they are logged less reliably, but we did not investigate this. The target (Fig. 5) is right-skewed: median 1.33 min, 5th–95th percentiles 0.50–4.65 min, and about 6.3% of intervals lie above the IQR upper fence. These long intervals look like genuine delays or dwell times, so they were kept and no winsorisation was applied to the target; the metrics are therefore sensitive to them (Section 6). In the correlation matrix (Fig. 6) no single variable has a strong linear relationship with the interval (largest |r| = 0.13, for hour of day), which anticipates the moderate R^2 reported later.

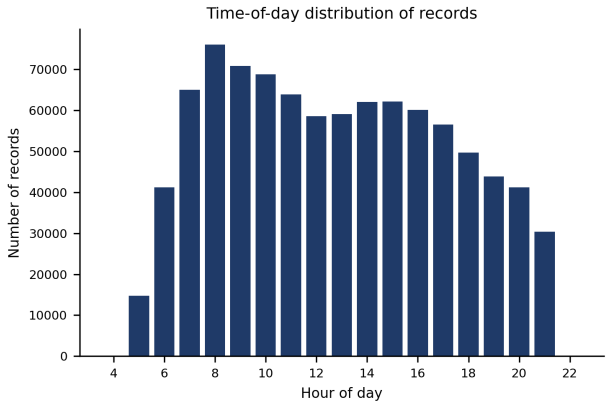


Fig. 3. Number of records by hour of day (all months and both folders).

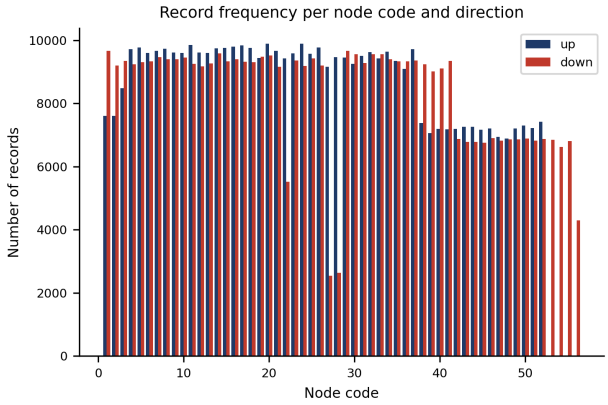


Fig. 4. Record frequency per node code and direction. Codes above 41 occur only in folder 534.

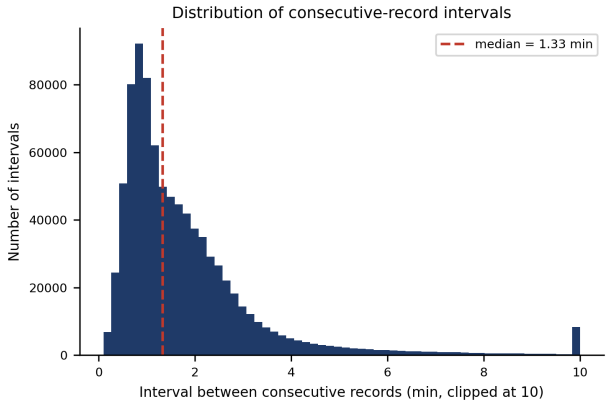


Fig. 5. Distribution of consecutive-record intervals (clipped at 10 min for display; the dashed line is the median).

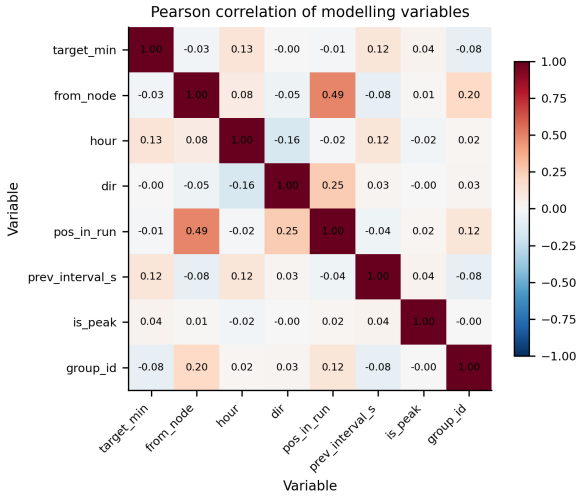


Fig. 6. Pearson correlation matrix of the target and the main numeric variables (all months, n = 867,574).

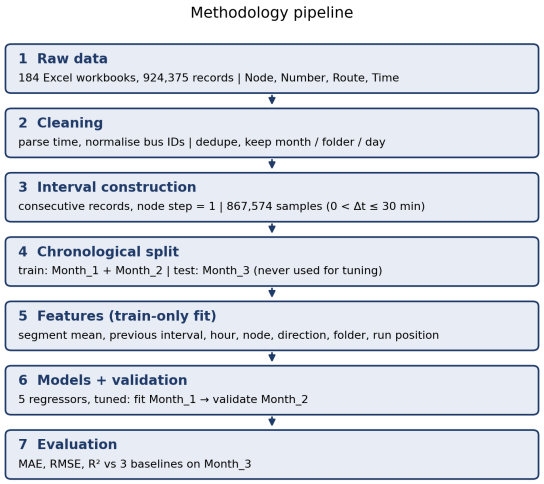


Fig. 7. Methodology pipeline. Everything after the split is fitted on training data only.

3.4 Feature engineering

For each sample the following features are available at prediction time (Table 3): the from-node code, direction, route-group folder, cyclic hour-of-day encoding (sin, cos of 2πt/86400 s), a peak-hour flag (08–10 h and 17–19 h, a choice we made without verifying Delhi's peak periods), the number of consecutive adjacent records already seen in the current run, the previous interval of the same bus (median-imputed when unavailable, with a missing-indicator; 6.0% of samples), and the historical mean interval of the same segment (folder, direction, from-node) computed on the training data only. Features are standardised for the models that need it, with the scaler fitted on training data inside each pipeline.

Table 3. Features used by the models (10 in total).

Feature	Description
from_node	node code at the start of the segment
dir, group_id	direction (up = 1); folder (425 / 534)
hour_sin, hour_cos, is_peak	cyclic time of day; peak-hour flag
pos_in_run	adjacent records already seen in this run
prev_interval_min, prev_missing	previous interval (imputed); missing flag
seg_hist_mean	training-set mean interval of the segment

4 Methodology

Fig. 7 summarises the workflow. Let **x** be the feature vector of a segment and *y* the interval in minutes. Three baselines are used: the training mean, the segment historical mean, and persistence (the previous interval of the same bus).

Linear regression fits $\hat{y} = \mathbf{w}^{T}\mathbf{x} + b$ by least squares. **Polynomial regression** expands the standardised features to all degree-2 terms (squares and pairwise products) before the same fit. **k-NN regression** averages the targets of the k nearest training points under Euclidean distance on standardised features. The **decision tree** recursively splits the feature space to minimise squared error within nodes, with maximum depth and minimum leaf size as controls. The **MLP** has two ReLU hidden layers, an L2 penalty α, early stopping on a 10% validation split and the Adam optimiser. Performance is measured by

$$MAE = (1/n)\,\Sigma\,|y - \hat{y}|,\quad RMSE = \sqrt{[(1/n)\,\Sigma\,(y - \hat{y})^2]},\quad R^2 = 1 - \Sigma\,(y - \hat{y})^2 / \Sigma\,(y - \bar{y})^2 \qquad (1)$$

5 Experimental Setup

Split. Data are split by month folder, not at random: training = Month_1 + Month_2 (585,678 samples), test = Month_3 (281,896). Month_3 was never used for tuning or for fitting any feature statistic. The files contain no calendar dates. Month_3 is treated as a held-out folder, but the folder labels do not establish a calendar sequence or a seasonal comparison. **Tuning.** Hyper-parameters were chosen by fitting on a 120,000-sample random subset of Month_1 and minimising RMSE on a 100,000-sample subset of Month_2; the selected settings were then refitted on all training data (Table 4). **Environment.** Python 3, scikit-learn, pandas, NumPy and Matplotlib; random seed 42. The complete code regenerates every number and figure in this paper (run_all.bat on Windows or python src/main.py); the README lists the steps and the order of the scripts.

Table 4. Hyper-parameter grids, selected settings and validation RMSE (min; fit Month_1, validate Month_2 subset).

Model	Grid searched	Selected	Val. RMSE
Linear Regression	none	default	1.296
Polynomial (deg 2)	degree 2	deg2	1.268
k-NN	k □ {15, 50, 100}	k=50	1.255
Decision Tree	depth {6,10,14} × leaf {50,200}	depth=14,leaf=50	1.246
MLP	layers {(64,32),(128,64)} × α {1e-4, 1e-2}	h=(128, 64),a=0.01	1.232

6 Results

6.1 Model comparison

Table 5 and Fig. 8 show that all five learned models beat the three baselines on every metric. The MLP is best (MAE 0.596, RMSE 1.426, R^2 0.474), but the decision tree (RMSE 1.428) and k-NN (1.432) are practically tied with it; the differences among these three are at most 0.006 min in RMSE, much smaller than the typical error, and we did not test their statistical significance. The MLP needed 686 s to fit against 2.8 s for the tree, so the tree is the better practical choice here. The linear model (R^2 0.422) and the degree-2 model (0.442) are somewhat weaker, which suggests mild non-linearity (for example, interactions between node and time of day).

Table 5. Test-set results on Month_3 (n = 281,896). MAE and RMSE in minutes; fit time in seconds on one CPU core. Best model in bold.

Model	MAE	RMSE	R^2	Fit (s)
Training mean	1.112	1.967	-0.001	0.0
Segment hist. mean	0.686	1.533	0.392	0.0
Persistence	1.422	2.513	-0.634	0.0
Linear	0.676	1.495	0.422	0.3
Polynomial (deg 2)	0.644	1.469	0.442	7.5
k-NN (k = 50)	0.599	1.432	0.470	36.8
Decision tree	0.598	1.428	0.472	2.8
MLP (128-64)	0.596	1.426	0.474	686.3

The strongest baseline is the segment historical mean (RMSE 1.533); the best model improves on it by 7.0% in RMSE and 13.1% in MAE. The gain in MAE is larger than in RMSE because a few very long intervals dominate squared error and none of the models can anticipate them. Persistence performs worst (R^2 -0.634): presumably because a single previous interval is a noisy proxy for the next one, and a negative R^2 means it is worse than always predicting the mean.

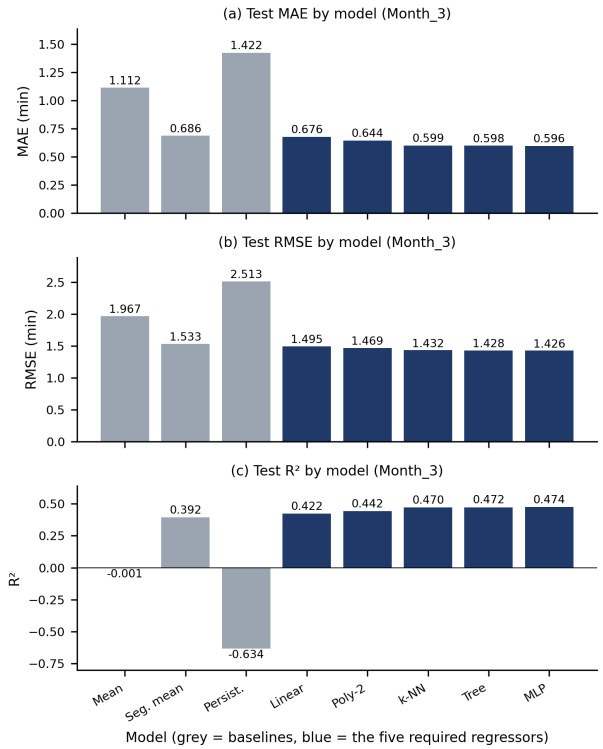


Fig. 8. Test (a) MAE, (b) RMSE and (c) R^2 of the three baselines (grey) and the five regressors (blue) on Month_3.

Table 6. Interval statistics (min) by month folder; the test month has the largest spread.

Month	Samples	Mean	Median	Std. dev.
Month 1	294,575	1.826	1.333	1.850
Month 2	291,103	1.849	1.333	1.804
Month 3	281,896	1.882	1.333	1.966

The test error is higher than the validation error used for tuning (RMSE 1.232 on the Month_2 subset against 1.426 on Month_3; R^2 0.528 against 0.474). Table 6 shows a likely contributing factor: the spread of the target grows from Month_1 to Month_3 (standard deviation 1.80 in Month_2, 1.97 in Month_3), so the last month is somewhat harder. Differences in subsampling may also play a role; we did not separate the two effects.

6.2 Error analysis

For the MLP, 71% of test predictions are within 0.5 min and 87% within 1 min of the observed interval; the median absolute error is 0.26 min but the 95th percentile is 2.02 min. The mean residual is +0.08 min, i.e. a slight average under-prediction. Fig. 9 shows good agreement for intervals below about 3 min and a wide scatter above them: very long observed intervals are rarely predicted. The residual plot (Fig. 10) shows the same asymmetry; its sharp lower edge is a property of the data, because an interval cannot be shorter than about 0.1 min, so the residual cannot fall below $-\hat{y}$.

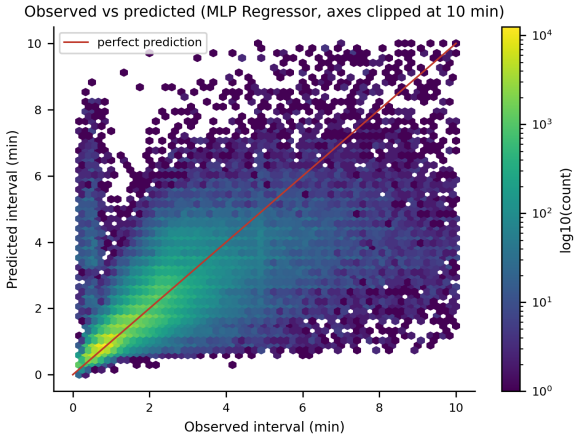


Fig. 9. Observed versus predicted interval for the MLP on the test month (density of points, axes clipped at 10 min).

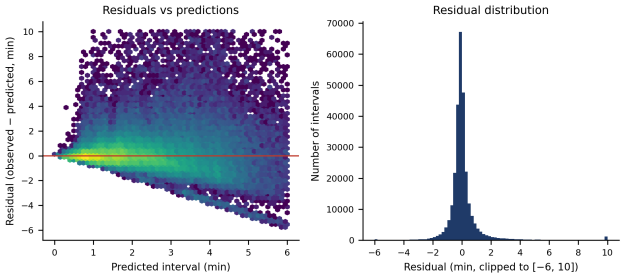


Fig. 10. Residuals of the MLP: (left) against the predicted value; (right) histogram (clipped to −6…10 min).

Error also depends on the time of day (Fig. 11): the MAE rises from 0.23 min in the early morning to 0.95 min at 19:00 and falls again after the evening peak. We did not investigate the cause (for example traffic, boarding time or fewer samples); the pattern should be read as descriptive only.

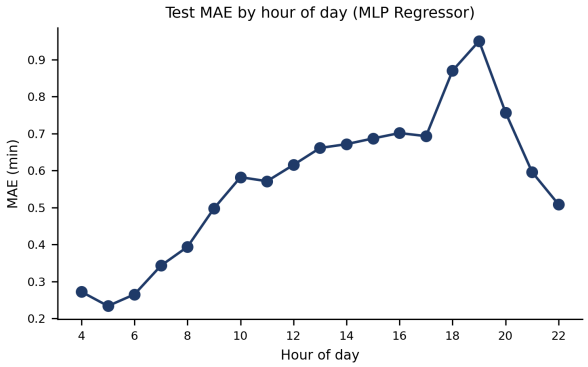


Fig. 11. Test MAE of the MLP by hour of day.

6.3 Feature importance and ablation

To see which inputs matter, we used the decision tree because it is fast enough to refit several times (the MLP takes 11 min). Permutation importance on a 60,000-sample subset of the test month (Fig. 12) is dominated by the segment historical mean (RMSE rises by 0.93 min when it is shuffled), followed by the hour-of-day features, the previous interval and the from-node code; direction, folder, run position and the peak flag matter little. Importance of correlated features is shared and should be read with care.

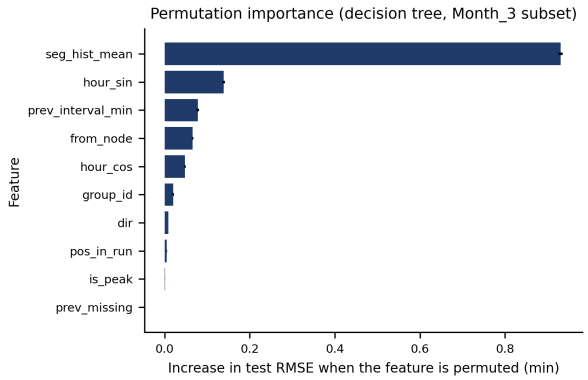


Fig. 12. Permutation importance of the ten features for the decision tree on a subset of the test month (error bars: standard deviation over three repeats).

The ablation in Table 7 gives a complementary view. Removing the hour-of-day features hurts most (RMSE 1.481 against 1.428 with all features). Removing the segment historical mean costs less (1.460) than its permutation importance suggests, because the tree can rebuild much of it from node, direction and folder; the same holds for the previous interval (1.445). Removing node/run position or direction/folder changes almost nothing. In short, time of day and segment identity carry most of the usable signal.

Table 7. Feature-group ablation for the decision tree (test month; "–" means the group is removed and the tree is refitted).

Setting	MAE	RMSE	R^2
All features	0.598	1.428	0.472
– segment historical mean	0.631	1.460	0.449
– previous interval (+flag)	0.605	1.445	0.460
– time of day (hour, peak)	0.646	1.481	0.433
– node / run position	0.600	1.432	0.470
– direction, folder	0.599	1.429	0.472

6.4 Error by folder and direction

For the MLP, errors differ across subgroups (Table 8). Folder 425 has the longer intervals (mean 2.19 min against 1.80 min) and larger errors (RMSE 1.63 against 1.37 min); the down direction is harder than the up direction (RMSE 1.62 against 1.20 min, R^2 0.42 against 0.55). A single overall score therefore hides differences between services, which matters if the model were used for passenger information.

Table 8. MLP test error by route-group folder and direction (MAE, RMSE in minutes).

Subgroup	n	MAE	RMSE	R^2
folder 425	57,164	0.752	1.629	0.556
folder 534	224,732	0.556	1.370	0.432
down	140,175	0.658	1.619	0.419
up	141,721	0.534	1.205	0.551

7 Discussion

What the results show. The available fields (segment, direction, folder, time of day and the previous interval) explain a little under half of the variance in interval length, and a historical segment average already captures most of this. More flexible models add a modest, consistent improvement. Because the three best models are so close, model choice should be guided by speed and simplicity rather than by the last decimal of RMSE.

Practical implications. For an operator, the simplest useful system would be a lookup of the historical mean interval by segment and time of day, refreshed regularly; the learned models improve on it by a small margin and mainly help on typical segments rather than on unusual delays. A decision tree gives most of the benefit with a fit time of seconds, can be retrained as new months arrive, and can be inspected, which is attractive for a transport department.

Limitations. (i) The target is the interval between consecutive *records*, not a verified stop-to-stop travel time or arrival time; node codes may be sequence indices rather than stops, and logging could have its own delays. (ii) The data hold no coordinates, schedule, weather or incident information, and no calendar dates, so weekday, holiday and seasonal effects cannot be modelled. (iii) The provenance of the data was not verified. (iv) The segment-mean feature is computed on all training rows, including each row's own target; with several thousand rows per segment the effect is small, but out-of-fold encoding would be cleaner. (v) Hyper-parameter search used subsamples and small grids, and results come from a single run with one seed. (vi) The test period follows the training period only by folder order, so we cannot say how well the models generalise to other seasons or years.

Ethics and SDG relevance. The data contain vehicle identifiers but no passenger information; even so, results should not be used to judge individual drivers. Reliable travel-time information would support SDG 11 (public transport as a practical alternative to private vehicles) and SDG 9 (data-driven transport infrastructure), but this study is a first step, not a deployable ETA system.

8 Conclusion and Future Work

Using 924,375 node-passing records of Delhi city buses, we built a reproducible pipeline and compared five classical regressors with three baselines on a held-out month. The best model (MLP) reached MAE 0.596 min and R^2 0.474, only slightly ahead of a decision tree and k-NN, and about 7% better in RMSE than a segment-wise historical mean. Future work should obtain a verified stop mapping and GPS-based arrival labels so that real stop-to-stop ETA can be predicted, add weather, calendar and schedule information, use gradient-boosted trees as an extra comparison, and test across more months.

Declarations

Data availability. The analysed files are the three supplied archives (Month_1, Month_2, Month_3; 184 workbooks). Their original source has not been independently confirmed and they are not redistributed with this paper. Code, figures and result tables are in the project folder.

Conflict of interest. The author declares no conflict of interest. **Funding.** None.

Use of AI tools. ChatGPT (OpenAI) was used for assistance with code troubleshooting and language editing. The dataset processing, model runs, numerical results and figures are drawn from the supplied project files and saved experiment outputs. The author is responsible for reviewing the analysis and confirming that the final paper accurately represents the work.

References

[1] N. Singh and K. Kumar, “A review of bus arrival time prediction using artificial intelligence,” *WIREs Data Mining and Knowledge Discovery*, vol. 12, no. 4, 2022, doi: 10.1002/widm.1457.

[2] A. Abdi and C. Amrit, “A review of travel and arrival-time prediction methods on road networks: classification, challenges and opportunities,” *PeerJ Computer Science*, vol. 7, Art. no. e689, 2021, doi: 10.7717/peerj-cs.689.

[3] A. A. Agafonov and A. S. Yumaganov, “Performance comparison of machine learning methods in the bus arrival time prediction problem,” in *Proc. ITNT-2019*, CEUR Workshop Proceedings, vol. 2416, 2019.

[4] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” *Information Sciences*, vol. 191, pp. 192–213, 2012, doi: 10.1016/j.ins.2011.12.028.

[5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[7] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[8] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015 (arXiv:1412.6980).

Appendix A. Validation tuning results

Validation scores (fit on a Month_1 subset, evaluated on a Month_2 subset); the test month was not used.

Model	Setting	Val. MAE	Val. RMSE	Val. R^2
Linear Regression	default	0.623	1.296	0.478
Polynomial (deg 2)	deg2	0.590	1.268	0.500
k-NN	k=15,uniform	0.566	1.264	0.503
k-NN	k=50,uniform	0.568	1.255	0.510
k-NN	k=100,uniform	0.577	1.264	0.503
Decision Tree	depth=6,leaf=50	0.588	1.284	0.487
Decision Tree	depth=6,leaf=200	0.589	1.285	0.487
Decision Tree	depth=10,leaf=50	0.565	1.249	0.515
Decision Tree	depth=10,leaf=200	0.573	1.261	0.506
Decision Tree	depth=14,leaf=50	0.562	1.246	0.517
Decision Tree	depth=14,leaf=200	0.571	1.258	0.508
MLP	h=(64, 32),a=0.0001	0.566	1.237	0.524
MLP	h=(64, 32),a=0.01	0.567	1.240	0.521
MLP	h=(128, 64),a=0.0001	0.563	1.234	0.527
MLP	h=(128, 64),a=0.01	0.556	1.232	0.528

Appendix B. Test error by hour of day

MLP test mean absolute error by hour. The same hourly MAE values are shown in Fig. 11.

Hour	MAE (min)
04:00	0.272
05:00	0.234
06:00	0.265
07:00	0.343
08:00	0.394
09:00	0.498
10:00	0.582
11:00	0.571
12:00	0.616
13:00	0.661
14:00	0.672
15:00	0.687
16:00	0.702
17:00	0.693
18:00	0.870
19:00	0.950
20:00	0.757
21:00	0.596
22:00	0.508

Appendix C. Reproducibility

The project folder contains *src/* (load_data, preprocess, eda, train_models, make_result_figures, make_pipeline_figure, extra_analysis, paper_stats, build_pdf, build_docx, main), *outputs/figures* (all figures as 300-dpi PNG), *outputs/tables* (model_comparison.csv, dataset_profile.csv, hyperparameter_tuning.csv, ablation and importance tables) and the paper in PDF and DOCX form. After placing the three unzipped Month folders in *data/*, running *run_all.bat* (Windows) or *python src/main.py* repeats every step; the MLP stage is the slowest (about 11 min on one CPU core). Random seed 42 is fixed for sampling, trees and the MLP.